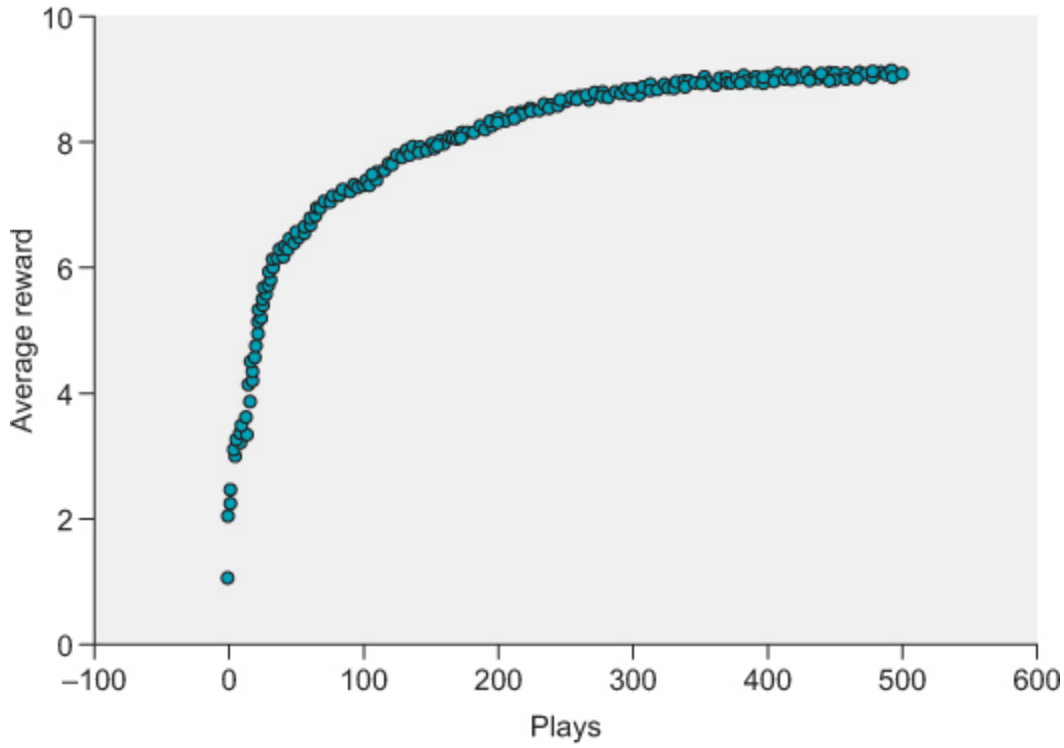
- ***3*** Chooses the best action with 0.8 probability, or randomly otherwise
- ***4*** Computes the reward for choosing the arm
- ***5*** Updates the record array with the new count and reward observation for this arm
- ***6*** Keeps track of the running average of rewards to assess overall performance

As you can see in figure 2.3, the average reward does indeed improve after many plays. Our algorithm is *learning*; it is getting reinforced by previous good plays! And yet it is such a simple algorithm.

Figure 2.3. This plot shows that the average reward for each slot machine play increases over time, indicating we are successfully learning how to solve the n-armed bandit problem.



The problem we've considered here is a *stationary* problem because the underlying reward probability distributions for the arms does not change over time. We certainly could consider a variant of this problem where this is not true—a nonstationary problem. In this case, a simple modification would be to allow new reward observations to update the average reward value stored in the record in a skewed way, so that it would be a weighted